TECHNICAL SPECIFICATION COMPANION | ENGINEERING BASELINE

WHISK & LAYERS: TECHNICAL APPENDIX

Detailed API Contracts, Relational Schema DDL, Traceability Matrix, Edge Cases & System Architecture

This document serves as the comprehensive engineering and architecture companion to the main Product Requirements Document ('Whisk_and_Layers_PRD_Final.docx'). It preserves deep technical specifications, full REST API contracts, entity-relationship schemas, comprehensive edge-case handling rules, design tokens, and source code mapping.

Technical Dimension	Specification / System Property
Document Identifier	APP-WL-2026-FINAL (Companion to PRD-WL-2026-FINAL)
Target Audience	Technical Product Managers, Engineering Leads, Full-Stack Architects, Hiring Reviewers
Monorepo Workspaces	backend (Node.js/Express/TypeScript) + frontend (React 18/Vite 6/TypeScript)
Database Engine	Universal Relational SQL Engine (SQLite with disk persistence in data/whisk_layers.sqlite)
Test Suite	Vitest 9/9 Integration Tests Passed (npm.cmd run test --workspace=backend)
Live Demo Architecture	Static bundle on GitHub Pages using client-side mockAdapter.ts fallback; zero remote backend required for evaluation.

TECHNICAL IMPLEMENTATION REALITY NOTICE: This appendix documents genuine technical implementation details from the Whisk & Layers codebase. It distinguishes verified code implementations and local automated test passes from mock demo behaviors, unvalidated market hypotheses, and future production roadmap requirements. No claims of unverified cloud infrastructure, production scale, or live payment gateway integration are made.

1. Detailed End-to-End User Flows & Interaction Logic

Flow 1: Customer Onboarding & JWT Authentication

- **Trigger:** User clicks 'Sign In' or enters checkout unauthenticated.
- **Execution Steps:** 1. User navigates to /register or /login. 2. Provides email, password (>= 6 chars), full name, 10-digit Indian phone (+91). 3. Submits form; frontend sends POST /api/auth/register or /login. 4. Backend validates with Zod; verifies/generates bcrypt hash. 5. Backend signs JWT with payload { userId, email, role }. 6. Frontend stores token in localStorage ('wl_token') and dispatches AuthContext state update.
- **Outcome & State:** User authenticated; user session loaded; redirected to origin page.

Flow 2: Bakery Discovery & Geospatial / Category Filtering

- **Trigger:** Customer seeks local artisan bakeries.
- **Execution Steps:** 1. User navigates to /bakeries. 2. Frontend triggers GET /api/bakeries. 3. User selects City (Ahmedabad, Mumbai, Rajkot, Surat, Vadodara) and Lead Time filter (<= 2 days, <= 3 days). 4. Frontend filters cards by city and minimum_lead_days. 5. User clicks bakery card to view storefront details and signature cakes.
- **Outcome & State:** Bakery details and catalog loaded via GET /api/bakeries/:idOrSlug.

Flow 3: Pre-Built Signature Cake Catalog Selection

- **Trigger:** Customer browses pre-designed cakes.
- **Execution Steps:** 1. User navigates to /cakes. 2. Frontend triggers GET /api/cakes with category/price query parameters. 3. User views high-res imagery, preparation lead days, eggless badges, and base price. 4. User inputs celebration plaque inscription (optional). 5. Clicks 'Add to Cart'.
- **Outcome & State:** Item appended to CartContext; single-bakery cart rule validated.

Flow 4: Interactive 8-Step Custom Cake Studio Flow

- **Trigger:** Customer wants to design a bespoke cake from scratch.
- **Execution Steps:** 1. User navigates to /custom-builder. 2. System fetches options via GET /api/customizations/options. 3. Configuration Step 1 (Architecture): Selects Tiers (1 to 4) and Shape (Round, Heart, Square, Hexagon). 4. Step 2 (Colors): Selects Frosting Palette (10 presets or hex picker) and Accent Piping color. 5. Step 3 (3D Garnishes): Toggles top toppings (Macarons, Berries, Gold Flecks, Candle, Flowers) rendered live onto Three.js canvas. 6. Step 4 (Sponge): Selects crumb sponge (Vanilla, Dutch Chocolate, Ruby Cocoa, Kesar Elaichi). 7. Step 5 (Filling): Selects gourmet compote (Belgian Truffle, Rasmalai, Biscoff, Alphonso Mango). 8. Step 6 (Dietary): Selects 100% Eggless, Vegan, Gluten-Free, or Low Sugar. 9. Step 7 (Inscription): Inputs custom lettering (max 65 chars). 10. Step 8 (Bakery Choice): Selects neighborhood artisan bakery. 11. Review / Blueprint Step: Reviews live Blueprint Summary Card showing itemized price and calculated delivery lead date. 12. Clicks 'Add Bespoke Cake to Cart'.
- **Outcome & State:** Custom cake snapshot saved in cart with 3D parameter properties.

Flow 5: Cart Management & Single-Bakery Lead Enforcement

- **Trigger:** Customer updates quantities or adds cake from another bakery.
- **Execution Steps:** 1. If item belongs to different bakery, modal prompts: 'Clear cart to switch bakeries?'. 2. Cart calculates max lead days across all items. 3. User adjusts quantities; subtotal updates.
- **Outcome & State:** Cart state synchronized in localStorage ('wl_cart').

Flow 6: Indian Address & Date Selection Checkout

- **Trigger:** Customer proceeds from cart to checkout.
- **Execution Steps:** 1. Navigates to /checkout. 2. Calendar disables dates earlier than today + max(bakery.minimum_lead_days, item.leadDays). 3. Customer selects 3-hour delivery slot. 4. Enters 10-digit Indian phone and 6-digit postal PIN code address. 5. Selects payment method (UPI QR, Card, NetBanking, COD).
- **Outcome & State:** Checkout form validated via frontend Zod rules.

Flow 7: Order Placement & Database Persistence Flow

- **Trigger:** Customer clicks 'Place Order'.
- **Execution Steps:** 1. Frontend sends POST /api/orders with bearer token. 2. orderService verifies lead days and re-calculates subtotal, ₹80 delivery fee, and 5% GST. 3. Inserts row into orders and order_items tables. 4. Generates unique order_number (e.g., 'WL-2026-8941'). 5. Inserts notification row for bakery owner.
- **Outcome & State:** Order created with status 'PENDING'; customer routed to confirmation page.

Flow 8: Bakery Order Review & Acceptance Flow

- **Trigger:** Bakery logs into dashboard to review incoming orders.
- **Execution Steps:** 1. Bakery visits /bakery/orders?status=PENDING. 2. Inspects order card: recipe snapshot, delivery date, time slot, inscription text. 3. Verifies kitchen capacity. 4. Clicks 'Accept Order'. 5. Backend executes PATCH /api/orders/:id/status { status: 'ACCEPTED' }.
- **Outcome & State:** Order status updated to 'ACCEPTED'; notification created for customer.

Flow 9: Bakery Order Rejection Flow (Mandatory Reason)

- **Trigger:** Bakery cannot fulfill order due to capacity/logistics.
- **Execution Steps:** 1. Bakery clicks 'Decline Order'. 2. System displays mandatory rejection modal. 3. Bakery types reason (>= 5 chars). 4. Backend validates rejectionReason via Zod. 5. Status updated to 'REJECTED' with rejection_reason stored.
- **Outcome & State:** Order terminated; customer notified with exact rejection reason.

Flow 10: Stage-by-Stage Order Milestone Tracking Flow

- **Trigger:** Bakery progresses production; customer checks timeline.
- **Execution Steps:** 1. Bakery clicks 'Start Baking' -> PREPARING. 2. Bakery clicks 'Mark Boxed & Ready' -> READY. 3. Bakery clicks 'Dispatch Order' -> OUT_FOR_DELIVERY. 4. Driver delivers; bakery marks -> DELIVERED.
- **Outcome & State:** Customer visual timeline advances; delivery unlocks review eligibility.

Flow 11: Bakery Cake Catalog Management

- **Trigger:** Bakery manages menu offerings and pricing.
- **Execution Steps:** 1. Bakery navigates to /bakery/cakes. 2. Adds new cake or edits existing item. 3. Updates base price in INR, image URL, preparation days, or toggles active status. 4. Backend persists updates via cakeRepository.
- **Outcome & State:** Public catalog immediately reflects updated pricing and availability.

Flow 12: Verified Customer Review & Merchant Reply

- **Trigger:** Customer receives delivered cake and rates experience.
- **Execution Steps:** 1. Order detail page displays 'Write a Verified Review' button. 2. Customer selects 1 to 5 stars and writes comment. 3. Backend validates status === 'DELIVERED' and order_id uniqueness. 4. Review saved; bakery rating_avg recalculated. 5. Bakery views review and posts public merchant reply.
- **Outcome & State:** Verified review and public bakery reply published on storefront.

2. Complete REST API Specifications & Request/Response Contracts

Standard Envelope: All endpoints return a consistent JSON structure: { success: boolean, data?: T, message?: string, error?: string }.

Method & Path	Access Role	Request Payload Summary	Success Response & Status	Error Codes
POST /api/auth/register	Public	{ email, password, fullName, phone, role ('CUSTOMER'|'BAKERY'), bakeryName?, postalCode? }	201 Created: { token, user: { id, email, fullName, role } }	400 (Email taken), 422 (Zod validation error)
POST /api/auth/login	Public	{ email, password }	200 OK: { token, user: { id, email, fullName, role, phone, avatarUrl }, bakeryId? }	401 (Invalid credentials), 422 (Zod validation error)
GET /api/auth/me	Authenticated	None (Bearer JWT in Authorization header)	200 OK: { user: { id, email, fullName, role, ... } }	401 (Missing/expired token)
PUT /api/auth/profile	Authenticated	{ fullName?, phone?, bio?, dietaryPreferences?, savedAddresses?, celebrationDates? }	200 OK: { user: { ...updatedUser } }	401 (Unauthorized), 422 (Validation error)
GET /api/bakeries	Public	Query params: ?city=&search=&limit=&page=	200 OK: { bakeries: Bakery[], total: number }	500 (Database error)
GET /api/bakeries/:idOrSlug	Public	Path param: bakery ID or unique slug	200 OK: { bakery: Bakery, cakes: Cake[], reviews: Review[] }	404 (Bakery not found)
PUT /api/bakeries/my-bakery	Bakery Only	{ name?, tagline?, description?, address?, city?, state?, postalCode?, phone?, minimumLeadDays? }	200 OK: { bakery: Bakery }	401 (Unauthorized), 403 (Not a bakery)
GET /api/cakes	Public	Query params:	200 OK: { cakes: Cake[], total:	500 (Database error)

		?category=&bakeryId=&minPrice=&maxPrice=&search=	number }	
POST /api/cakes	Bakery Only	{ name, description, basePrice, categoryId, preparationDays, imageUrl, isCustomizable? }	201 Created: { cake: Cake }	401 (Unauthorized), 403 (Forbidden), 422 (Invalid input)
GET /api/customizations/options	Public	None	200 OK: { options: CustomOption[], grouped: { BASE, FLAVOR, SIZE, SHAPE, ICING, TOPPING, DECORATION } }	500 (Database error)
POST /api/orders	Customer Only	{ bakeryId, customerName, customerPhone, deliveryAddress, deliveryDate, deliveryTimeSlot, items: OrderItem[] }	201 Created: { order: Order (with order_number, subtotal, delivery_fee, tax_amount, total_amount) }	400 (Lead time error, inactive bakery), 401, 422
GET /api/orders/my-orders	Customer Only	None (Bearer JWT)	200 OK: { orders: Order[] (with items, bakery name) }	401 (Unauthorized)
GET /api/orders/:id	Authenticated	Path param: order ID	200 OK: { order: Order (with items, reviews) }	401 (Unauthorized), 403 (Not own order), 404 (Not found)
GET /api/orders/bakery/dashboard	Bakery Only	None (Bearer JWT)	200 OK: { pendingCount, preparingCount, completedCount, totalRevenue, urgentOrders: Order[] }	401 (Unauthorized), 403 (Forbidden)
GET /api/orders/bakery/orders	Bakery Only	Query params: ?status=&page=&limit=	200 OK: { orders: Order[], total: number }	401 (Unauthorized), 403 (Forbidden)
PATCH /api/orders/:id/status	Bakery / Admin	{ status: OrderStatus, rejectionReason? }	200 OK: { order: Order (status updated) }	400 (Invalid transition), 403 (Wrong bakery), 422 (Missing rejectionReason)
POST /api/reviews	Customer Only	{ orderId, rating (1-5), comment }	201 Created: { review: Review }	400 (Order not delivered, already reviewed), 401, 422
POST /api/reviews/:id/reply	Bakery Only	{ replyComment }	200 OK: { review: Review (with reply_comment) }	401 (Unauthorized), 403 (Not bakery owner of review)
GET /api/admin/metrics	Admin Only	None (Bearer JWT with role='ADMIN')	200 OK: { totalUsers, totalBakeries, totalOrders, totalGmv, recentOrders: Order[] }	401 (Unauthorized), 403 (Forbidden: non-admin)

3. Universal Relational Data Model & Schema DDL Specification

The database architecture utilizes standard ANSI SQL, implemented via SQLite in local development and compatible with enterprise MySQL / PostgreSQL migrations. The DDL below represents the active schema in backend/src/database/schema.ts:

```
-- 1. USERS TABLE
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  full_name TEXT NOT NULL,
  phone TEXT,
  role TEXT NOT NULL CHECK(role IN ('CUSTOMER', 'BAKERY', 'ADMIN')),
  avatar_url TEXT,
  bio TEXT,
  dietary_preferences TEXT,
  saved_addresses TEXT,
  celebration_dates TEXT,
  is_active INTEGER NOT NULL DEFAULT 1,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 2. BAKERIES TABLE
CREATE TABLE bakeries (
```

```
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL UNIQUE,
  name TEXT NOT NULL,
  slug TEXT UNIQUE NOT NULL,
  tagline TEXT,
  description TEXT NOT NULL,
  address TEXT NOT NULL,
  city TEXT NOT NULL,
  state TEXT NOT NULL,
  postal_code TEXT NOT NULL,
  phone TEXT NOT NULL,
  email TEXT NOT NULL,
  logo_url TEXT,
  banner_url TEXT,
  specialties TEXT,
  rating_avg REAL DEFAULT 0.0,
  review_count INTEGER DEFAULT 0,
  is_approved INTEGER NOT NULL DEFAULT 1,
  is_active INTEGER NOT NULL DEFAULT 1,
  minimum_lead_days INTEGER NOT NULL DEFAULT 2,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- 3. CAKE CATEGORIES & CAKES TABLES
CREATE TABLE cake_categories (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL UNIQUE,
  slug TEXT NOT NULL UNIQUE,
  description TEXT,
  image_url TEXT,
  sort_order INTEGER DEFAULT 0
);

CREATE TABLE cakes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  bakery_id INTEGER NOT NULL,
  category_id INTEGER,
  name TEXT NOT NULL,
  slug TEXT NOT NULL,
  description TEXT NOT NULL,
  base_price REAL NOT NULL,
  preparation_days INTEGER NOT NULL DEFAULT 2,
  image_url TEXT NOT NULL,
  gallery_urls TEXT,
  is_customizable INTEGER NOT NULL DEFAULT 0,
  is_available INTEGER NOT NULL DEFAULT 1,
  is_active INTEGER NOT NULL DEFAULT 1,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (bakery_id) REFERENCES bakeries(id) ON DELETE CASCADE,
  FOREIGN KEY (category_id) REFERENCES cake_categories(id) ON DELETE SET NULL
);

-- 4. CUSTOMIZATION OPTIONS TABLE
CREATE TABLE customization_options (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  type TEXT NOT NULL CHECK(type IN ('BASE', 'FLAVOR', 'SIZE', 'SHAPE', 'ICING', 'TOPPING', 'DECORATION')),
  label TEXT NOT NULL,
  description TEXT,
  extra_price REAL NOT NULL DEFAULT 0.0,
  image_url TEXT,
  is_active INTEGER NOT NULL DEFAULT 1,
  sort_order INTEGER DEFAULT 0
);
```

```
-- 5. ORDERS & ORDER ITEMS TABLES
CREATE TABLE orders (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  order_number TEXT UNIQUE NOT NULL,
  customer_id INTEGER NOT NULL,
  bakery_id INTEGER NOT NULL,
  total_amount REAL NOT NULL,
  subtotal REAL NOT NULL,
  delivery_fee REAL NOT NULL DEFAULT 0.0,
  tax_amount REAL NOT NULL DEFAULT 0.0,
  status TEXT NOT NULL DEFAULT 'PENDING' CHECK(status IN ('PENDING', 'ACCEPTED', 'PREPARING', 'READY',
'OUT_FOR_DELIVERY', 'DELIVERED', 'REJECTED', 'CANCELLED')),
  rejection_reason TEXT,
  customer_name TEXT NOT NULL,
  customer_phone TEXT NOT NULL,
  delivery_address TEXT NOT NULL,
  delivery_date TEXT NOT NULL,
  delivery_time_slot TEXT,
  special_instructions TEXT,
  payment_method TEXT NOT NULL DEFAULT 'PAY_ON_DELIVERY',
  payment_status TEXT NOT NULL DEFAULT 'PENDING' CHECK(payment_status IN ('UNPAID', 'PENDING', 'PAID',
'REFUNDED')),
  transaction_id TEXT,
  payment_details TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (customer_id) REFERENCES users(id) ON DELETE RESTRICT,
  FOREIGN KEY (bakery_id) REFERENCES bakeries(id) ON DELETE RESTRICT
);

CREATE TABLE order_items (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  order_id INTEGER NOT NULL,
  cake_id INTEGER,
  cake_name TEXT NOT NULL,
  cake_image TEXT,
  base_price REAL NOT NULL,
  quantity INTEGER NOT NULL DEFAULT 1,
  subtotal REAL NOT NULL,
  is_custom INTEGER NOT NULL DEFAULT 0,
  custom_message TEXT,
  selected_options TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
  FOREIGN KEY (cake_id) REFERENCES cakes(id) ON DELETE SET NULL
);

-- 6. REVIEWS & NOTIFICATIONS TABLES
CREATE TABLE reviews (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  order_id INTEGER UNIQUE NOT NULL,
  customer_id INTEGER NOT NULL,
  bakery_id INTEGER NOT NULL,
  rating INTEGER NOT NULL CHECK(rating >= 1 AND rating <= 5),
  comment TEXT NOT NULL,
  reply_comment TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
  FOREIGN KEY (customer_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (bakery_id) REFERENCES bakeries(id) ON DELETE CASCADE
);

CREATE TABLE notifications (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  type TEXT NOT NULL,
  title TEXT NOT NULL,
```

```
  message TEXT NOT NULL,
  link_url TEXT,
  is_read INTEGER NOT NULL DEFAULT 0,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

4. Functional Requirements Traceability Matrix

Every functional requirement is mapped to its implementing frontend components, backend controllers, services, repositories, database tables, and verification tests:

Req ID	Functional Capability	Frontend Component	Backend Controller / Service	Database Tables	Verification Test / Status
FR-001	User Authentication & JWT	LoginPage.tsx, RegisterPage.tsx	authController.ts, authService.ts	users	Vitest Test 2, 3, 4 (Verified)
FR-002	Role-Based Access Control	RequireRole.tsx, RequireAuth.tsx	authMiddleware.ts (requireRole)	users.role	Vitest Test 5 (Verified)
FR-003	Bakery Discovery & Filtering	BakeriesPage.tsx, BakeryDetailPage.tsx	bakeryController.ts, bakeryRepo.ts	bakeries, reviews	Vitest Test 6 (Verified)
FR-004	Cake Catalog & Details	CakesPage.tsx, CakeDetailPage.tsx	cakeController.ts, cakeRepo.ts	cakes, cake_categories	Vitest Test 6 (Verified)
FR-005	Interactive 8-Step Studio	CustomCakeBuilderPage.tsx	customizationController.ts	customization_options	Vitest Test 6 (Verified)
FR-006	Real-Time 3D Visualizer	Cake3DViewer.tsx (Three.js WebGL)	Client-side WebGL engine	N/A (Client graphics)	Observed in Demo (Verified)
FR-007	Cart Isolation & Max Lead	CartContext.tsx, CartPage.tsx	Client Context & orderService.ts	localStorage ('wl_cart')	Vitest Test 7 (Verified)
FR-008	Address & Slot Checkout	CheckoutPage.tsx	orderController.ts, orderService.ts	orders	Vitest Test 7, 9 (Verified)
FR-009	Order Placement & Price Lock	CheckoutPage.tsx, OrderConfirm.tsx	orderController.ts, orderService.ts	orders, order_items	Vitest Test 7 (Verified)
FR-010	Bakery Order Acceptance	BakeryOrdersPage.tsx, Dashboard.tsx	orderController.ts, orderService.ts	orders.status	Vitest Test 7 (Verified)
FR-011	Mandatory Rejection Logging	DeclineOrderModal.tsx	orderController.ts, schemas.ts	orders.rejection_reason	Vitest Test 8 (Verified)
FR-012	Fulfillment Milestone Tracking	OrderDetailPage.tsx, Timeline.tsx	orderController.ts, notifRepo.ts	orders, notifications	Vitest Test 7 (Verified)
FR-013	Verified Reviews & Replies	ReviewModal.tsx, BakeryReviews.tsx	reviewController.ts, reviewRepo.ts	reviews	Vitest Test 7 (Verified)
FR-014	Storefront & Catalog Management	BakeryProfilePage.tsx, CakesPage.tsx	bakeryController.ts, cakeController.ts	bakeries, cakes	Verified in Code
FR-015	Platform Governance Metrics	AdminDashboardPage.tsx	adminController.ts, adminRepo.ts	Platform aggregate SQL	Vitest Test 4 (Verified)

5. Comprehensive Edge Cases & Exception Handling Matrix

Case ID	Scenario Description	Trigger Condition	Implemented System Defense	Failure Mode Prevented
EC-001	Multi-Bakery Cart Addition	User adds cake from Bakery B while cart holds Bakery A.	CartContext prompts modal: 'Clear cart to add from new bakery?'.	Multi-bakery delivery routing & split payment failure.
EC-002	Lead Time Date Violation	Customer manipulates date picker to select tomorrow for a 3-day cake.	Backend orderService re-checks deliveryDate >= today + leadDays; rejects 400.	Baker forced to cancel due to impossible curing window.
EC-003	Order Rejection Without Reason	Bakery attempts to PATCH status to REJECTED with empty string.	Zod validation catches rejectionReason.min(5); returns HTTP 422.	Customer receives opaque cancellation with zero explanation.

EC-004	Review Before Delivery	Customer attempts to POST /api/reviews while order is PREPARING.	Backend verifies order.status === 'DELIVERED'; returns HTTP 400.	Fake or premature reviews distorting marketplace ratings.
EC-005	Duplicate Review Submission	Customer clicks 'Submit Review' twice rapidly or retries POST.	Database enforces UNIQUE(order_id) constraint on reviews table.	Artificially inflated review counts for merchants.
EC-006	Negative Quantity or Price Injection	Malicious client sends { quantity: -2, basePrice: -500 } in payload.	Zod schema validates quantity.min(1) and basePrice.positive(); rejects 422.	Negative order totals and revenue calculation corruption.
EC-007	SQL Injection in Custom Notes	Attacker submits `'; DROP TABLE orders; --` in custom inscription.	All SQL queries use parameterized queries (prepared statements via db.ts).	Database destruction or arbitrary SQL execution.
EC-008	Oversized Inspiration Image	User uploads raw 40MB DSLR photo as custom reference.	Frontend rejects files > 8MB; backend Multer enforces 8MB limits.	Server storage exhaustion and memory crash.
EC-009	Unauthorized Dashboard Access	Customer crafts request to /api/orders/bakery/dashboard with valid token.	authMiddleware checks user.role === 'BAKERY'; rejects with HTTP 403.	Customer viewing competitor financials and merchant data.
EC-010	Network Disconnect on Payment	Customer connection drops while submitting checkout.	Order remains uncommitted; client catches error and prompts safe retry.	Ghost orders created without confirmation visibility.

6. Design System Tokens & WebGL 3D Visualization Engine

6.1 Design System Color & Typography Tokens

Defined in frontend/src/styles/index.css, the warm artisan design system reflects bakery aesthetics:

Token CSS Variable	Hex Value	Semantic Role & Usage Context
--primary	#C93B67 (Plum Berry) / #9B2C2C (Deep Maroon)	Primary brand CTA buttons, active state highlights, accents.
--primary-light	#FDF2F4 / #FFF8F6	Active tab backgrounds, card selection highlights, soft badges.
--bg-main	#FDFBF7 (Artisan Flour Cream)	Main application page background; warm organic canvas.
--bg-card	#FFFFFF (Pure Snow White)	Card containers, modals, forms, and elevation surfaces.
--text-main	#1E293B (Deep Charcoal Slate)	Primary headings, titles, high-contrast readable UI body text.
--text-muted	#64748B (Slate Muted Gray)	Secondary text, metadata, lead time badges, ingredient descriptions.
--border-light	#E2E8F0 / #ECE0D1	Subtle card dividers, input borders, structural table lines.
--font-serif	'Playfair Display', Georgia, serif	Artisan headings (h1, h2, h3), celebration cake titles.
--font-sans	'Plus Jakarta Sans', system-ui, sans-serif	Navigation, buttons, form controls, tables, body text.

6.2 Three.js WebGL 3D Visualization Engine Architecture

Implemented in frontend/src/components/ui/Cake3DViewer.tsx, the interactive 3D visualizer provides real-time tactile feedback without external asset loading lag:

- **Procedural Geometry Engine:** Stack dynamically builds 1 to 4 tiers based on user selection. Supports 4 base geometries: CylinderGeometry for Classic Round, custom Shape/ExtrudeGeometry for Sweetheart, BoxGeometry with rounded corners for Crisp Square, and CylinderGeometry with 6 radial segments for Regal Hexagon.
- **Dynamic Materials & Shading:** Tier meshes utilize MeshStandardMaterial with configurable roughness (0.4) and metalness (0.05). Primary frosting color and accent piping hex values update material uniforms in real-time.
- **Procedural Garnish Positioning:** 3D garnishes (French Macarons, Exotic Berries, 24K Gold Flecks, Lit Birthday Candle, Botanical Petals) are procedurally attached to the cake crown using local coordinate offsets. The celebration candle includes a pulsating PointLight and particle flame shader.
- **Camera & Orbit Controls:** Three.js OrbitControls permit full 360-degree azimuthal rotation and polar pitch clamping (15 to 85 degrees) to prevent disorienting camera angles.

7. Future Engineering Architecture & Feasibility Evaluations

7.1 Live Razorpay / Cashfree UPI Gateway Integration Architecture

Phase 2 requires replacing simulated checkout modals with a licensed Indian payment gateway (Razorpay or Cashfree):

- **1. Order Creation:** Client requests POST /api/payments/create-order; backend initiates gateway order via Razorpay Node SDK with currency INR; returns order_id.
- **2. Payment Execution:** Razorpay Checkout JS opens native UPI intent (GPay, PhonePe, Paytm). Upon completion, gateway sends signed payment_id and signature.
- **3. Signature Verification:** Backend verifies HMAC-SHA256 signature using razorpay_secret. Order transitions to 'PENDING_CONFIRMATION'.
- **4. Automated Refund Webhook:** If bakery declines order, backend automatically triggers payment refund API (/v1/payments/:id/refund) and logs reversal in transactions table.

7.2 Automated WhatsApp Business API Architecture

To ensure high delivery rates for time-sensitive notifications, the system will integrate WhatsApp Cloud API (or Gupshup):

- **Template Approvals:** Approved HSM templates for 'New Order Received', 'Order Accepted', 'Cake Dispatched', and 'Order Rejection Notice'.
- **Event-Driven Worker Queue:** orderService triggers async webhook dispatch on status changes; worker queue (BullMQ + Redis) handles delivery retries.

7.3 Hyperlocal Logistics Dispatch API (Dunzo / Shadowfax API)

Phase 3 scale requires automated refrigerated courier dispatch:

- **1. Automated Dispatch Call:** When order moves to 'READY', system pings 3PL dispatch API with pickup PIN, delivery PIN, thermal box requirement, and customer phone.
- **2. Rider Assignment & GPS Tracking:** Courier accepts assignment; webhook receives rider name, phone, and GPS coordinate tracking URL, rendered in customer order timeline.